CARLOS EXPEDITO RODRIGUES ARAGÃO

POS TECH

IA PARA DEVS

PROCESSAMENTO DE LINGUAGEM NATURAL

AULA 04

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

SUMÁRIO

O QUE VEM POR AÍ?.....3
HANDS ON 4
SAIBA MAIS.....12
O QUE VOCÊ VIU NESTA AULA? 18
REFERÊNCIAS.....19

FIAP

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

O QUE VEM POR AÍ?

Nesta aula você vai continuar melhorando o nosso modelo de classificação, implementando técnicas poderosas de Stemming, TF IDF e Ngrams, que nos ajudará a melhorar nossa acurácia.

FIAP

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

HANDS ON

Continuaremos analisando o nosso classificador de sentimento, porém implementando técnicas de Stemmização com a biblioteca NLTK, vetorização com TF IDF e aplicação de Ngrams nos tokens.

Primeiro, aplicamo stemmização nos nossos dados.

```
frase_processada = list()
for avaliacao in avaliacoes.texto_sem_stopwords_e_pontuacao_e_acentos_minusculo:
  nova_frase = list()
  palavras_texto = token_pontuacao.tokenize(avaliacao)
  for palavra in palavras_texto:
    if palavra not in stopwords_sem_acento:
      nova_frase.append(stemmer.stem(palavra))
  frase_processada.append(' '.join(nova_frase))



avaliacoes["texto_stemmizado"] = frase_processada
```

Vai demorar um pouquinho para rodar porque esse processo de stemmizacao é mais demorado. Comparando a acurácia do modelo com a última coluna e nossa nova coluna com o texto stemmizado, temos uma mehora significativa:

```
treinar_modelo(avaliacoes,
"texto_sem_stopwords_e_pontuacao_e_acentos_minusculo", "polarity")
```

```
treinar_modelo(avaliacoes, "texto_stemmizado", "polarity")
```

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

Visualizando as word clouds, percebemos que agora temos nosso texto sem sufixos.

```
word_cloud_neg(avaliacoes, "texto_stemmizado")
```

Figura 1- Nuvem de palavras (1)
Fonte: Elaborado pelo autor (2024)

```
word_cloud_pos(avaliacoes, "texto_stemmizado")
```

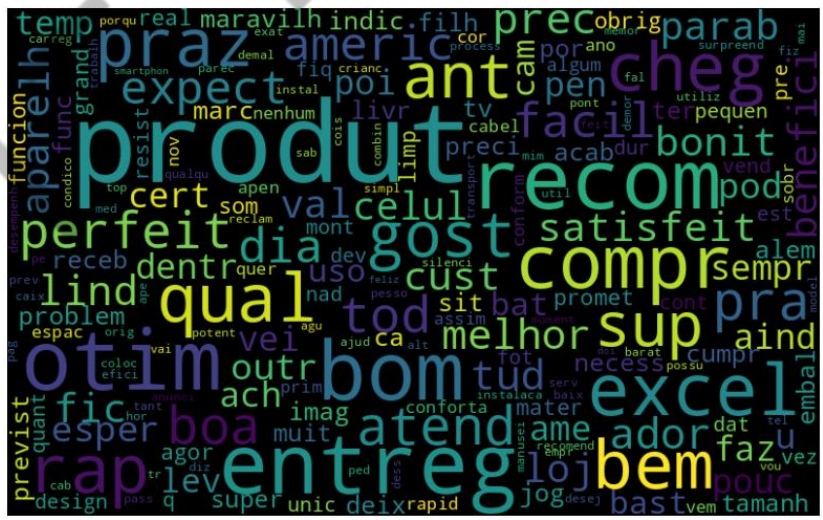

Figura 2 – Nuvem de palavras (2)
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

No nosso gráfico também.

```
grafico(avaliacoes, "texto_stemmizado", 10)
```

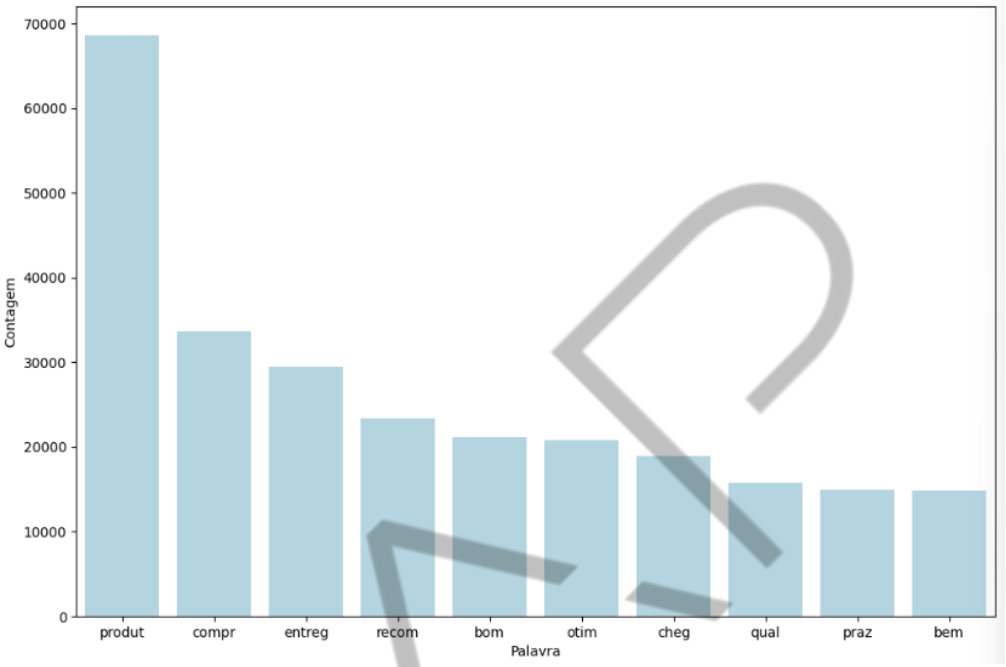

Figura 3 – Gráfico de palavras
Fonte: Elaborado pelo autor (2024)

Após isso, nós implementamos o TF IDF e fizemos um exemplo para entender como funciona.

```
from sklearn.feature_extraction.text import TfidfVectorizer


frases = ["Este produto é muito bom", "Este produto é muito ruim"]

tfidf = TfidfVectorizer(lowercase=False, max_features=100)

caracteristicas = tfidf.fit_transform(frases)


pd.DataFrame(

    caracteristicas.todense(),

    columns=tfidf.get_feature_names_out()
```

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

```
)
```

	Este	bom	muito	produto	ruim
0	0.448321	0.630099	0.448321	0.448321	0.000000
1	0.448321	0.000000	0.448321	0.448321	0.630099

E aplicamos isso no nosso modelo para verificar a acurácia.

```
tfidf_tratados = tfidf.fit_transform(avaliacoes.texto_stemmizado)

treino, teste, classe_treino, classe_teste = train_test_split(tfidf_tratados,
                                                              avaliacoes.polarity,
                                                              random_state = 71)


regressao_logistica = LogisticRegression()

regressao_logistica.fit(treino, classe_treino)

acuracia_tfidf = regressao_logistica.score(teste, classe_teste)

acuracia_tfidf
```

Após isso, nós adicionamos a implementação do Ngrams com um exemplo também.

```
from nltk import ngrams


frase = "Comprei um ótimo produto."

frase_separada = token_por_espaço.tokenize(frase)

pares = ngrams(frase_separada, 2)

list(pares)
```

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

```
[('Comprei', 'um'), ('um', 'ótimo'), ('ótimo', 'produto.')]
```

E aplicamos no nosso modelo.

```
tfidf = TfidfVectorizer(lowercase=False, ngram_range = (1,2))
vetor_tfidf = tfidf.fit_transform(avaliacoes.texto_stemmizado)
treino, teste, classe_treino, classe_teste = train_test_split(vetor_tfidf,
                                                              avaliacoes.polarity,
                                                              random_state = 71)

regressao_logistica = LogisticRegression()
regressao_logistica.fit(treino, classe_treino)
acuracia_tfidf_ngrams = regressao_logistica.score(teste, classe_teste)
acuracia_tfidf_ngrams
```

Assim, obtendo uma acurácia, mas como estávamos fazendo a medição com as 100 primeiras palavras do modelo, nós vamos rodar a base sem essa restrição e sem o Ngrams e verificar a diferença.

```
tfidf = TfidfVectorizer(lowercase=False)
vetor_tfidf = tfidf.fit_transform(avaliacoes.texto_stemmizado)
treino, teste, classe_treino, classe_teste = train_test_split(vetor_tfidf,
                                                              avaliacoes.polarity,
                                                              random_state = 71)

regressao_logistica = LogisticRegression()
regressao_logistica.fit(treino, classe_treino)
acuracia_tfidf = regressao_logistica.score(teste, classe_teste)
```

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

```
acuracia_tfidf
```

Com isso você pode notar uma melhora na sua análise.

Analisamos também os pesos que a própria regressão logística monta a partir do TF IDF e vamos ver quais palavras foram mais importantes para a diferenciação de sentenças positivas ou negativas.

Positivas:

```
pesos = pd.DataFrame(
    regressao_logistica.coef_[0].T,
    index = tfidf.get_feature_names_out()
)

pesos.nlargest(10, 0)
```

	0
otim	18.167819
excel	16.946000
bom	13.041082
perfeit	11.380505
ador	10.228067
satisfeit	8.754339
rap	8.606541
lind	8.467364
ame	7.405624
maravilh	7.300235

Figura 4 – Lista de palavras positivas
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

Negativas:

```
pesos.nsmallest(10, 0)
```

	0
pess	-10.538815
ruim	-8.878490
frac	-7.536810
receb	-7.474489
decepcion	-6.964363
horri	-6.652518
fragil	-6.367967
dinh	-6.272407
vei	-6.118027
devolv	-5.997585

Figura 5 – Lista de palavras negativas
Fonte: Elaborado pelo autor (2024)

Ao analisar os 50 primeiros nós podemos observar alguns bigramas que foram importantes para a análise.

```
pesos.nlargest(50, 0)
```

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

tud	4.737651
surpreend	4.700695
ant	4.652845
produt otim	4.631170
obrig	4.626784
produt bom	4.542732
bonit	4.518275
sempr	4.402408
melhor	4.339074
produt excel	4.287887
top	4.239390
exat	4.151841
rapid	4.067893
otim produt	3.992060
necess	3.973490
amou	3.935145
feliz	3.834420
util	3.765979
efici	3.750767
gost produt	3.692453
praz	3.667250
sup recom	3.628300
atend	3.550801
conforta	3.429960

Figura 6 – Bigramas
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

SAIBA MAIS

A stemmização é um processo crucial no campo do Processamento de Linguagem Natural (PLN), que visa simplificar a análise de texto. Esse método tem como propósito reduzir palavras às suas formas fundamentais, conhecidas como "stems". A motivação por trás do uso de stemming reside na variação morfológica inerente a um idioma, onde palavras podem apresentar diferentes formas derivadas, como plurais, conjugações verbais, tempos verbais, entre outras.

O objetivo principal da stemmização é a redução da dimensionalidade do espaço vocabular, facilitando assim tarefas de PLN. Além disso, ao agrupar variações morfológicas sob uma única representação, o processo de stemming contribui para aumentar a precisão em diversas tarefas, incluindo recuperação de informações, busca textual e classificação de documentos.

Existem várias técnicas de stemming, mas a abordagem mais comum envolve a remoção de sufixos das palavras, preservando apenas a raiz ou o núcleo. Diversos algoritmos foram desenvolvidos para implementar essa técnica, sendo alguns dos mais populares o Porter Stemmer, o Snowball Stemmer e o Lancaster Stemmer. Cada algoritmo possui suas próprias regras e estratégias para realizar a stemmização.

Um exemplo prático de stemming pode ser observado ao aplicar o processo à palavra "running". Após a stemmização, a palavra é reduzida à sua forma básica "run". Esse exemplo ilustra como o stemming pode agrupar palavras relacionadas sob uma forma comum, simplificando a análise semântica.

Contudo, é importante reconhecer as limitações da stemmização, como o overstemming, onde palavras diferentes são reduzidas a uma única forma, e o understemming, que mantém palavras distintas com formas similares. Além disso, em alguns casos, a stemmização pode levar à perda de informações semânticas.

No universo da PLN, o uso de stemming, com algoritmos como o Porter Stemmer e Snowball Stemmer, desempenha um papel fundamental, contribuindo para a eficiência e a precisão de diversas aplicações, desde motores de busca até sistemas de análise de sentimentos. Ao compreender a teoria e a implementação prática da stemmização, pessoas desenvolvedoras podem aprimorar suas soluções e explorar todo o potencial dessa técnica linguística.

FIAP

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

O RSLP (Removedor de Sufixos da Língua Portuguesa) é um algoritmo de stemmização desenvolvido para a língua portuguesa. Assim como outros algoritmos de stemmização, o RSLP tem como objetivo principal reduzir palavras às suas formas fundamentais, chamadas "stems" ou radicais, removendo sufixos morfológicos.

Este algoritmo foi proposto por Valerio de Carvalho em sua dissertação de mestrado em 2007 e é projetado especificamente para lidar com a complexidade morfológica do português. A língua portuguesa apresenta desafios únicos em comparação com o inglês, por exemplo, devido a sua rica inflexão verbal, conjugações e flexões gramaticais.

O RSLP utiliza uma abordagem baseada em regras para realizar a stemmização. Cada regra é cuidadosamente projetada para lidar com diferentes contextos morfológicos, removendo sufixos e transformando as palavras para suas formas fundamentais. O algoritmo é capaz de lidar com uma variedade de casos, incluindo substantivos, verbos, adjetivos e advérbios.

Embora o RSLP seja uma opção popular para a stemmização em português, é importante notar que nenhum algoritmo de stemmização é perfeito, e a stemmização em si possui limitações. O RSLP pode apresentar casos de understemming (quando diferentes palavras são reduzidas à mesma forma) ou overstemming (quando palavras diferentes não são reduzidas o suficiente). A eficácia do RSLP pode variar dependendo do contexto específico e do tipo de texto que está sendo processado.

Em aplicações de Processamento de Linguagem Natural (PLN) voltadas para o português, o RSLP é uma ferramenta valiosa para simplificar o tratamento de palavras e melhorar a eficiência em tarefas como recuperação de informações, análise de sentimentos e classificação de documentos. Ao considerar o RSLP, os desenvolvedores e desenvolvedoras podem ajustar e adaptar a stemmização conforme necessário para atender às exigências específicas de seus projetos.

O Term Frequency-Inverse Document Frequency (TF-IDF) é uma técnica avançada e amplamente utilizada no campo de Processamento de Linguagem Natural (PLN) que desempenha um papel crucial na avaliação da importância relativa das palavras em documentos. Ao combinar duas medidas distintas, Term Frequency (TF) e Inverse Document Frequency (IDF), o TF-IDF oferece uma abordagem sofisticada para representar e ponderar termos em uma coleção de documentos.

FIAP

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

O primeiro componente, TF, busca avaliar a frequência de uma palavra específica em um determinado documento. Essencialmente, o TF mede a proporção do número de vezes que um termo aparece em um documento pelo número total de termos naquele documento. Isso destaca termos que ocorrem com maior frequência em um contexto específico, proporcionando uma representação relativa à importância local.

Por sua vez, o IDF concentra-se em avaliar a raridade de um termo em toda a coleção de documentos. A fórmula IDF utiliza o logaritmo do inverso da proporção do número total de documentos na coleção pelo número de documentos que contêm o termo mais um. Dessa forma, termos menos frequentes na coleção, mas presentes em documentos específicos, recebem um peso mais alto, contribuindo para a identificação de termos mais distintivos.

A combinação dessas duas medidas, TF e IDF, resulta no cálculo do TF-IDF. Multiplicando o TF pelo IDF, a técnica cria uma representação ponderada que destaca palavras que são frequentes em um documento específico, mas raras na coleção como um todo. Isso é particularmente valioso em tarefas como recuperação de informações, onde a relevância de um documento para uma consulta específica é fundamental.

O TF-IDF encontra aplicações em uma variedade de cenários, incluindo motores de busca, classificação de documentos, agrupamento de textos e mineração de texto. Sua capacidade de atribuir pesos distintos a termos com base em sua importância local e global o torna uma ferramenta versátil para análise de texto.

Contudo, é importante reconhecer as limitações do TF-IDF. Em documentos muito curtos, o cálculo do TF pode levar a representações enviesadas, e a técnica pode não capturar completamente nuances semânticas e contextuais das palavras.

Em resumo, o TF-IDF destaca-se como uma técnica robusta e eficaz no universo do PLN, proporcionando uma abordagem sofisticada para representação e ponderação de termos em documentos, contribuindo significativamente para a precisão e eficácia em diversas tarefas linguísticas.

A implementação do TF-IDF com o NLTK (Natural Language Toolkit) envolve uma série de etapas, desde o pré-processamento do texto até a aplicação das

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

fórmulas TF e IDF. Vou guiar você através do processo, destacando as principais etapas sem fornecer um código específico.

Pré-processamento do Texto: antes de calcular o TF-IDF, é necessário realizar o pré-processamento do texto. Isso inclui tokenização (dividir o texto em palavras), remoção de pontuações, conversão para minúsculas, remoção de stopwords (palavras comuns sem muito significado) e, possivelmente, lematização ou stemming para reduzir as palavras às suas formas básicas.

Construção do Vocabulário: o próximo passo é construir um vocabulário único a partir dos documentos da coleção. Isso geralmente envolve a criação de um conjunto de todas as palavras distintas encontradas nos documentos.

Cálculo do Term Frequency (TF): para cada documento na coleção, calcula-se o TF para cada termo no vocabulário. O TF mede a frequência relativa de um termo em um documento específico, conforme explicado anteriormente.

Cálculo do Inverse Document Frequency (IDF): calcula-se o IDF para cada termo no vocabulário. O IDF mede a raridade de um termo em toda a coleção de documentos.

Multiplicação de TF pelo IDF: multiplica-se o TF pelo IDF para cada termo em cada documento. Isso resulta no cálculo final do TF-IDF para cada termo em cada documento.

Representação Final: cada documento é representado por um vetor TF-IDF, onde cada elemento do vetor corresponde ao TF-IDF de um termo específico no vocabulário.

Aplicações: a representação TF-IDF pode ser utilizada em várias tarefas, como recuperação de informações, classificação de documentos, agrupamento de textos e até mesmo na identificação de palavras-chave em um corpus.

Ao utilizar o NLTK para implementar o TF-IDF, você pode aproveitar as funcionalidades dessa biblioteca para realizar tokenização, remoção de stopwords e outras etapas do pré-processamento. Além disso, você pode precisar incorporar outras bibliotecas, como o scikit-learn, para calcular o IDF e realizar operações matriciais eficientes.

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

Em resumo, a implementação do TF-IDF com o NLTK envolve uma combinação de etapas de pré-processamento, cálculos estatísticos e representação vetorial. A modularidade e a flexibilidade do NLTK tornam-no uma escolha conveniente para integrar essas etapas em seus projetos de processamento de linguagem natural.

N-grams são uma abordagem fundamental no campo do Processamento de Linguagem Natural (PLN), permitindo uma análise mais detalhada da estrutura e dos padrões linguísticos em documentos. Essa técnica envolve a subdivisão do texto em unidades consecutivas de N palavras ou caracteres, conhecidas como unigramas (N=1), bigramas (N=2), trigramas (N=3) e assim por diante. A expressão dessas sequências oferece insights valiosos sobre a ordem e a proximidade das palavras, revelando nuances semânticas e contextos linguísticos.

A aplicabilidade dos N-grams é vasta. Em modelos probabilísticos de linguagem, como o Modelo de Markov, a probabilidade de uma palavra depende das palavras anteriores, e os N-grams são essenciais nesse contexto. Além disso, eles desempenham um papel crucial no reconhecimento de entidades, correção ortográfica e sistemas de tradução automática.

A fórmula de cálculo de probabilidade para N-grams segue a regra da cadeia condicional, considerando a probabilidade de cada termo dado os anteriores. No entanto, para lidar com o problema da esparsidade, onde muitos N-grams podem não aparecer nos dados de treinamento, técnicas de suavização, como a suavização de Laplace, são comumente aplicadas.

Na análise de N-grams, a probabilidade condicional torna-se uma ferramenta valiosa para criar representações linguísticas mais ricas e contextuais. Por exemplo, ao analisar bigramas e trigramas, modelos podem capturar nuances de significado e relações sintáticas, aprimorando tarefas de Processamento de Linguagem Natural.

Contudo, os N-grams enfrentam desafios em lidar com longas dependências, onde a probabilidade condicional pode perder informatividade com a distância. Para superar essas limitações, modelos mais avançados, como Redes Neurais Recorrentes (RNNs), são explorados.

Em resumo, os N-grams são uma ferramenta poderosa e flexível em PLN, proporcionando insights profundos sobre padrões linguísticos e contextos semânticos. A compreensão aprofundada dos N-grams é essencial para explorar seu potencial na

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

análise de texto e na construção de modelos de linguagem mais robustos e contextualmente informados.

FIAP

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Você aprendeu como implementar o NLTK aos conceitos de stemming retirando os sufixos das palavras, o TF IDF normalizando os tokens e definindo pesos para que se tornem modelos mais precisos. Vimos ainda que os Ngrams são uma abordagem fundamental no campo do Processamento de Linguagem Natural e implementamos ele na análise trazendo informações sequenciais para os tokens.

FIAP

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

REFERÊNCIAS

NLTK. **Documentação RSLP.** [s.d]. Disponível em: <https://www.nltk.org/_modules/nltk/stem/rslp.html>. Acesso em: 17 jan. 2024.

ORENGO, V. HUYCK, C. **RSLP Stemmer (Removedor de Sufixos da Lingua Portuguesa).** 2001. Disponível em: <https://www.inf.ufrgs.br/~viviane/rslp/index.htm> Acesso em: 17 jan. 2024.

SKLEARN. **Documentação SkLearn TfidfVectorizer**. [s.d]. Disponível em: <https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html> Acesso em: 17 jan. 2024.

FIAP

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

PALAVRAS-CHAVE

Stemming. TF IDF. Ngrams.

FIAP

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com

PDF exclusivo para WILSON SILVA CUNHA - rm370006
wylson@gmail.com